Volume: Advanced Microcontrollers & IoT with ESP32

Section: 4.3: Connecting to the Internet

Topic: Creating a Wi-Fi Access Point (AP) and Connecting to a Network (STA), Simple Web Server for Remote Control

1. Introduction to ESP32 Wi-Fi Modes

The ESP32 is a versatile microcontroller with built-in Wi-Fi capabilities, making it ideal for Internet of Things (IoT) applications. It can operate in several Wi-Fi modes, primarily **Station (STA) mode**, **Access Point (AP) mode**, or a combination of both (**AP_STA mode**).

- **Station (STA) Mode**: In this mode, the ESP32 acts as a client, connecting to an existing Wi-Fi network (like your home router). This allows the ESP32 to access the internet and communicate with other devices on the network.
- **Access Point (AP) Mode**: In this mode, the ESP32 creates its own Wi-Fi network, allowing other devices (like smartphones or computers) to connect directly to it. This is useful for creating standalone devices that can be configured or controlled locally without an existing Wi-Fi infrastructure.
- **AP_STA Mode**: The ESP32 can simultaneously operate as both an Access Point and a Station, connecting to an existing network while also hosting its own network.

This module will focus on setting up the ESP32 in both AP and STA modes, and then demonstrate how to create a simple web server for remote control.

2. ESP32 in Station (STA) Mode: Connecting to a Wi-Fi Network

In STA mode, the ESP32 connects to an existing Wi-Fi network. This is the most common mode for IoT devices that need to send data to the cloud or interact with other network devices.

2.1. Basic Connection Steps

To connect the ESP32 to a Wi-Fi network in STA mode using the Arduino IDE, you typically follow these steps:

1. Include the `WiFi.h` library.
2. Define the SSID (network name) and password of the Wi-Fi network.
3. In the `setup()` function, set the Wi-Fi mode to `WIFI_STA`.
4. Initiate the connection using `WiFi.begin(ssid, password)`.
5. Wait for the ESP32 to connect to the network.
6. Once connected, the ESP32 will obtain an IP address from the router.

2.2. Example Code: Connecting to Wi-Fi

Here’s a basic example of an ESP32 connecting to a Wi-Fi network:

```
#include <WiFi.h>

const char* ssid = "YOUR_SSID";         // Your Wi-Fi network SSID
const char* password = "YOUR_PASSWORD";     // Your Wi-Fi network password

void setup() {
  Serial.begin(115200);
  delay(10);

  Serial.print("Connecting to ");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA); // Set ESP32 to Station mode
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting...");
  }

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  // Your code here to interact with the internet or other network devices
}
```

Explanation:

- `WiFi.mode(WIFI_STA)` : Configures the ESP32 to operate as a Wi-Fi station.
- `WiFi.begin(ssid, password)` : Attempts to connect to the specified Wi-Fi network.
- `WiFi.status() != WL_CONNECTED` : This loop waits until the ESP32 successfully connects to the Wi-Fi network.
- `WiFi.localIP()` : Returns the IP address assigned to the ESP32 by the router.

2.3. Practice Problem

Problem: Modify the example code to connect to a different Wi-Fi network. If the connection fails after 10 attempts, print an error message and restart the ESP32.

Solution:

```
#include <WiFi.h>

const char* ssid = "NEW_SSID";        // New Wi-Fi network SSID
const char* password = "NEW_PASSWORD";    // New Wi-Fi network password

void setup() {
  Serial.begin(115200);
  delay(10);

  Serial.print("Connecting to ");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 10) {
    delay(1000);
    Serial.println("Connecting...");
    attempts++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("");
    Serial.println("WiFi connected");
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());
  } else {
    Serial.println("");
    Serial.println("Failed to connect to WiFi after 10 attempts.
Restarting...");
    ESP.restart(); // Restart the ESP32
  }
}

void loop() {
  // Your code here
}
```

3. ESP32 in Access Point (AP) Mode: Creating a Wi-Fi Network

In AP mode, the ESP32 creates its own Wi-Fi network, acting like a router. Other devices can connect to this network, allowing direct communication with the ESP32.

3.1. Basic AP Setup Steps

To set up the ESP32 as an Access Point using the Arduino IDE, you typically follow these steps:

1. Include the `WiFi.h` library.
2. Define the SSID and password for the AP network you want to create.
3. In the `setup()` function, set the Wi-Fi mode to `WIFI_AP`.
4. Start the AP using `WiFi.softAP(ssid, password)`.
5. Optionally, get and print the IP address of the AP.

3.2. Example Code: Creating an Access Point

Here's a basic example of an ESP32 creating an Access Point:

```
#include <WiFi.h>

const char* ap_ssid = "ESP32_AP";        // The SSID for your ESP32 Access
Point
const char* ap_password = "123456789";    // The password for your ESP32
Access Point

void setup() {
  Serial.begin(115200);
  delay(10);

  Serial.print("Setting up AP (Access Point)...");
  WiFi.softAP(ap_ssid, ap_password); // Start the Access Point

  IPAddress IP = WiFi.softAPIP(); // Get the IP address of the AP
  Serial.print("AP IP address: ");
  Serial.println(IP);
}

void loop() {
  // Your code here to handle connections from clients
}
```

Explanation:

- `WiFi.softAP(ap_ssid, ap_password)`: Starts the ESP32 in AP mode with the specified SSID and password.
- `WiFi.softAPIP()`: Returns the IP address of the ESP32 in AP mode. Devices connecting to this AP will use this IP address to communicate with the ESP32.

3.3. Practice Problem

Problem: Create an ESP32 Access Point with a different SSID and no password (open network). Print a message indicating that the AP is open.

Solution:

```
#include <WiFi.h>

const char* ap_ssid_open = "MyESP32_OpenAP"; // SSID for the open Access
Point

void setup() {
  Serial.begin(115200);
  delay(10);

  Serial.print("Setting up Open AP (Access Point)...");
  // Start the Access Point without a password (open network)
  WiFi.softAP(ap_ssid_open);

  IPAddress IP = WiFi.softAPIP();
  Serial.print("AP IP address: ");
  Serial.println(IP);
  Serial.println("This is an open Wi-Fi network.");
}

void loop() {
  // Your code here
}
```

4. Simple Web Server for Remote Control

Combining the Wi-Fi capabilities with a web server allows for remote control of the ESP32. A web server hosted on the ESP32 can serve a simple HTML page with buttons or input fields to control GPIOs or display sensor data.

4.1. Web Server Basics

To create a simple web server on the ESP32, you typically use the `WebServer` library (or handle HTTP requests manually with `WiFiServer`). The process involves:

1. Setting up the ESP32 in either STA or AP mode (or AP_STA).
2. Creating a `WiFiServer` object on a specific port (e.g., port 80 for HTTP).
3. Listening for incoming client connections.
4. When a client connects, read the incoming HTTP request.

5. Parse the request to determine the desired action (e.g., turn an LED on/off).
6. Send an HTML response back to the client to display the web page.

4.2. Example Code: Web Server in AP Mode to Control LEDs

This example demonstrates a web server running on an ESP32 in AP mode, allowing you to control two LEDs connected to GPIO 26 and GPIO 27 via a web browser.

```
#include <WiFi.h>

const char* ssid = "ESP32-Access-Point";
const char* password = "123456789";

WiFiServer server(80);

String header;

String output26State = "off";
String output27State = "off";

const int output26 = 26;
const int output27 = 27;

void setup() {
  Serial.begin(115200);
  pinMode(output26, OUTPUT);
  pinMode(output27, OUTPUT);
  digitalWrite(output26, LOW);
  digitalWrite(output27, LOW);

  Serial.print("Setting AP (Access Point)...");
  WiFi.softAP(ssid, password);
  IPAddress IP = WiFi.softAPIP();
  Serial.print("AP IP address: ");
  Serial.println(IP);

  server.begin();
}

void loop() {
  WiFiClient client = server.available();   // Listen for incoming clients

  if (client) {                             // If a new client connects,
    Serial.println("New Client.");
    String currentLine = "";
    while (client.connected()) {
      if (client.available()) {
        char c = client.read();
        Serial.write(c);
        header += c;
        if (c == '\n') {
          if (currentLine.length() == 0) {
            client.println("HTTP/1.1 200 OK");
```

```
            client.println("Content-type:text/html");
            client.println("Connection: close");
            client.println();

            if (header.indexOf("GET /26/on") >= 0) {
              Serial.println("GPIO 26 on");
              output26State = "on";
              digitalWrite(output26, HIGH);
            } else if (header.indexOf("GET /26/off") >= 0) {
              Serial.println("GPIO 26 off");
              output26State = "off";
              digitalWrite(output26, LOW);
            } else if (header.indexOf("GET /27/on") >= 0) {
              Serial.println("GPIO 27 on");
              output27State = "on";
              digitalWrite(output27, HIGH);
            } else if (header.indexOf("GET /27/off") >= 0) {
              Serial.println("GPIO 27 off");
              output27State = "off";
              digitalWrite(output27, LOW);
            }

            client.println("<!DOCTYPE html><html>");
            client.println("<head><meta name=\"viewport\"
content=\"width=device-width, initial-scale=1\">");
            client.println("<link rel=\"icon\" href=\"data:,\">");
            client.println("<style>html { font-family: Helvetica; display:
inline-block; margin: 0px auto; text-align: center;}");
            client.println(".button { background-color: #4CAF50; border:
none; color: white; padding: 16px 40px;");
            client.println("text-decoration: none; font-size: 30px; margin:
2px; cursor: pointer;}");
            client.println(".button2 {background-color: #555555;}</style>
</head>");

            client.println("<body><h1>ESP32 Web Server</h1>");

            client.println("<p>GPIO 26 - State " + output26State + "</p>");
            if (output26State=="off") {
              client.println("<p><a href=\"/26/on\"><button
class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/26/off\"><button class=\"button
button2\">OFF</button></a></p>");
            }
```

```
            client.println("<p>GPIO 27 - State " + output27State + "</p>");
            if (output27State=="off") {
              client.println("<p><a href=\"/27/on\"><button
class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/27/off\"><button class=\"button
button2\">OFF</button></a></p>");
            }
            client.println("</body></html>");

            client.println();
            break;
          } else {
            currentLine = "";
          }
        } else if (c != '\r') {
          currentLine += c;
        }
      }
    }
    header = "";
    client.stop();
    Serial.println("Client disconnected.");
    Serial.println("");
  }
}
```

Explanation:

- The ESP32 is configured as an Access Point, creating its own Wi-Fi network.
- A `WiFiServer` is initialized on port 80.
- In the `loop()`, the server listens for incoming client connections.
- When a client connects, the ESP32 reads the HTTP request header.
- Based on the URL in the request (e.g., `/26/on`), the corresponding GPIO is set HIGH or LOW.
- An HTML page is generated and sent back to the client, displaying the current state of the GPIOs and providing buttons to control them.

4.3. Practice Problem

Problem: Modify the web server example to display the current uptime of the ESP32 in seconds on the web page. The uptime should update automatically without requiring a page refresh.

Solution:

To achieve automatic updates without a full page refresh, you would typically use JavaScript and AJAX requests. However, for a simple example within the scope of this module, we'll focus on displaying the uptime on page load. Implementing AJAX would require a more advanced web server setup (e.g., using `ESPAsyncWebServer` library) which is beyond the scope of this beginner module.

```
#include <WiFi.h>

const char* ssid = "ESP32-Access-Point";
const char* password = "123456789";

WiFiServer server(80);

String header;

String output26State = "off";
String output27State = "off";

const int output26 = 26;
const int output27 = 27;

void setup() {
  Serial.begin(115200);
  pinMode(output26, OUTPUT);
  pinMode(output27, OUTPUT);
  digitalWrite(output26, LOW);
  digitalWrite(output27, LOW);

  Serial.print("Setting AP (Access Point)...");
  WiFi.softAP(ssid, password);
  IPAddress IP = WiFi.softAPIP();
  Serial.print("AP IP address: ");
  Serial.println(IP);

  server.begin();
}

void loop() {
  WiFiClient client = server.available();   // Listen for incoming clients

  if (client) {
    Serial.println("New Client.");
    String currentLine = "";
    while (client.connected()) {
      if (client.available()) {
        char c = client.read();
        Serial.write(c);
        header += c;
        if (c == '\n') {
          if (currentLine.length() == 0) {
            client.println("HTTP/1.1 200 OK");
```

```
            client.println("Content-type:text/html");
            client.println("Connection: close");
            client.println();

            if (header.indexOf("GET /26/on") >= 0) {
              Serial.println("GPIO 26 on");
              output26State = "on";
              digitalWrite(output26, HIGH);
            } else if (header.indexOf("GET /26/off") >= 0) {
              Serial.println("GPIO 26 off");
              output26State = "off";
              digitalWrite(output26, LOW);
            } else if (header.indexOf("GET /27/on") >= 0) {
              Serial.println("GPIO 27 on");
              output27State = "on";
              digitalWrite(output27, HIGH);
            } else if (header.indexOf("GET /27/off") >= 0) {
              Serial.println("GPIO 27 off");
              output27State = "off";
              digitalWrite(output27, LOW);
            }

            client.println("<!DOCTYPE html><html>");
            client.println("<head><meta name=\"viewport\"
content=\"width=device-width, initial-scale=1\">");
            client.println("<link rel=\"icon\" href=\"data:,\">");
            client.println("<style>html { font-family: Helvetica; display:
inline-block; margin: 0px auto; text-align: center;}");
            client.println(".button { background-color: #4CAF50; border:
none; color: white; padding: 16px 40px;");
            client.println("text-decoration: none; font-size: 30px; margin:
2px; cursor: pointer;}");
            client.println(".button2 {background-color: #555555;}</style>
</head>");

            client.println("<body><h1>ESP32 Web Server</h1>");

            // Display uptime
            client.print("<p>Uptime: ");
            client.print(millis() / 1000); // millis() returns milliseconds,
convert to seconds
            client.println(" seconds</p>");

            client.println("<p>GPIO 26 - State " + output26State + "</p>");
            if (output26State=="off") {
              client.println("<p><a href=\"/26/on\"><button
```

```
class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/26/off\"><button class=\"button button2\">OFF</button></a></p>");
            }

            client.println("<p>GPIO 27 - State " + output27State + "</p>");
            if (output27State=="off") {
              client.println("<p><a href=\"/27/on\"><button class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/27/off\"><button class=\"button button2\">OFF</button></a></p>");
            }
            client.println("</body></html>");

            client.println();
            break;
          } else {
            currentLine = "";
          }
        } else if (c != '\r') {
          currentLine += c;
        }
      }
    }
    header = "";
    client.stop();
    Serial.println("Client disconnected.");
    Serial.println("");
  }
}
```

5. Conclusion

This module has provided a foundational understanding of how to configure the ESP32 in both Station (STA) and Access Point (AP) modes, and how to implement a basic web server for remote control. These are fundamental concepts for developing a wide range of IoT applications with the ESP32.

6. References

[1] Random Nerd Tutorials. “ESP32 Access Point (AP) for Web Server”. https://randomnerdtutorials.com/esp32-access-point-ap-web-server/ [2] Keyestudio. “HOW to Use WIFI (Station Mode) with ESP32”. https://www.keyestudio.com/blog/how-to-use-wifi-station-mode-with-esp32-269